

Observabilidade de ponta a ponta

Métricas, traces e logs em Spring Boot, Kafka e Outbox

Do fundamento ao diagnóstico de falhas reais

Este tutorial registra a arquitetura construída durante o estudo e explica por que cada peça existe: Actuator, Micrometer, Prometheus, Grafana, OpenTelemetry, OTLP, Jaeger, Loki, Alloy, propagação W3C, Outbox, retries, DLT e idempotência.

MÉTRICAS

Prometheus + Grafana

TRACES

OpenTelemetry + Jaeger

LOGS

Loki + Alloy + Grafana

Casos estudados

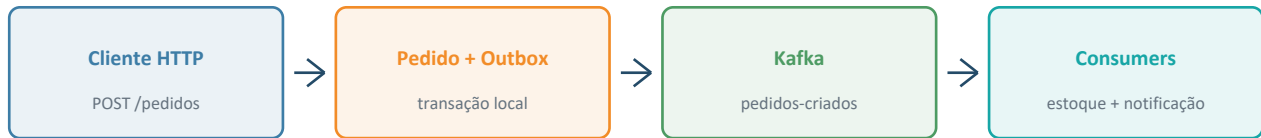
Fluxo normal · falha persistente 999.99 · retry + DLT · falha pós-commit 888.88

Idempotência do producer · idempotência do consumer · janela de duplicidade da Outbox

1 · MAPA DO TUTORIAL

O sistema que construímos

A observabilidade foi adicionada sem alterar a regra central: a API grava o pedido e o evento na mesma transação; um job publica a Outbox e dois consumers processam a mensagem.



Três sinais sobre o mesmo fluxo

Sinal	Pergunta que responde	Ferramenta no projeto
Métricas	Quanto? Com que frequência? A tendência piorou?	Micrometer → Prometheus → Grafana
Traces	Por onde a operação passou? Onde demorou ou falhou?	Micrometer Observation → OpenTelemetry/OTLP → Jaeger
Logs	Qual foi a narrativa detalhada e os valores envolvidos?	Logback → arquivo → Alloy → Loki → Grafana

Ideia-guia

As três fontes não competem. A métrica aponta o sintoma; o trace localiza a etapa; o log fornece o detalhe. traceid e eventid conectam a investigação.

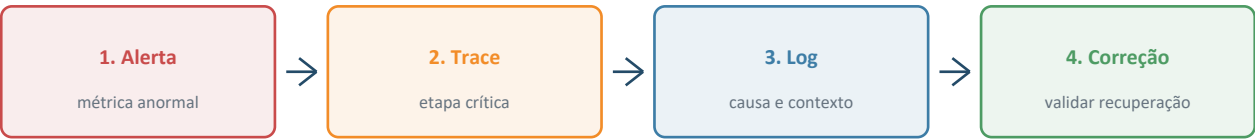
2 · FUNDAMENTOS

Observabilidade não é apenas monitoramento

Monitorar é acompanhar perguntas conhecidas. Observabilidade é conseguir investigar também perguntas que não foram previstas, usando os sinais emitidos pelo sistema.

Conceito	Foco	Exemplo no projeto
Monitoramento	Condição conhecida e alerta	Outbox pendente acima do normal
Observabilidade	Explicar o estado interno a partir das saídas	Descobrir se a demora ocorreu no job, Kafka ou consumer
Telemetria	Dados produzidos pelo sistema	Séries temporais, spans e linhas de log

Uma investigação típica



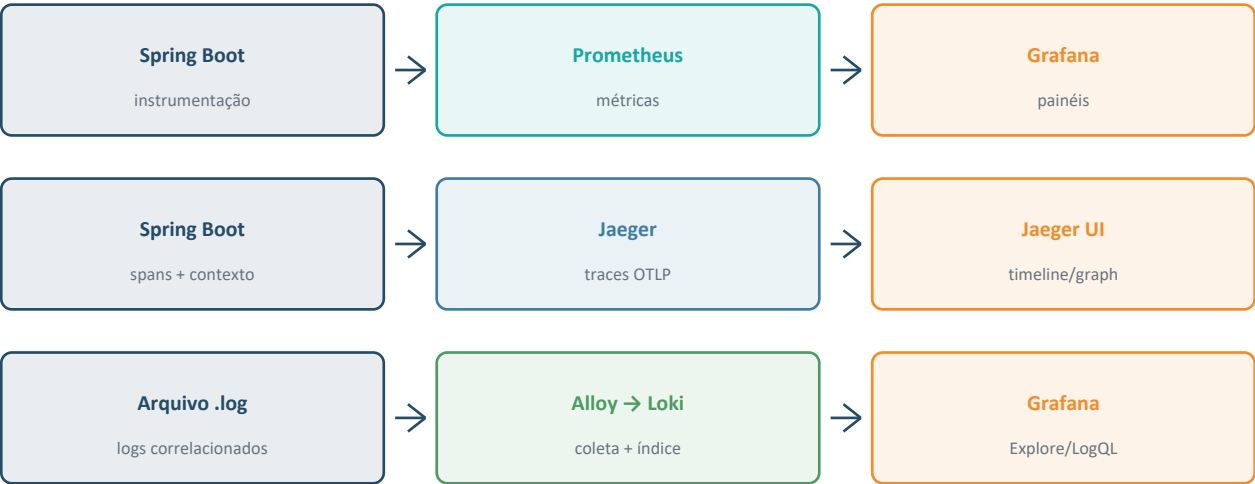
- Métricas são excelentes para agregação e tendência, mas não carregam identificadores únicos.
- Traces preservam causalidade entre etapas síncronas e assíncronas.
- Logs são flexíveis e detalhados, mas precisam de correlação e retenção controlada.

Regra de ouro
Não transforme tudo em métrica, tudo em span ou tudo em log. Escolha o sinal conforme a pergunta operacional.

3 · ARQUITETURA

A pilha implementada

Cada ferramenta tem uma responsabilidade clara. O Grafana é a janela de consulta; os dados continuam armazenados nos backends especializados.



Componente	Responsabilidade	Não confundir com
Actuator	Endpoints operacionais do Spring Boot	Backend de observabilidade
Micrometer	Fachada e API de observações	Banco que armazena métricas/traces
OpenTelemetry	Padrão, SDK e exportação de telemetria	Interface de consulta como o Jaeger
Grafana	Consulta e visualização de fontes	Armazenamento do Prometheus ou Loki

4 · SPRING BOOT

Actuator: a porta operacional

O Actuator adiciona endpoints de produção para saúde, métricas e integração com sistemas externos. No projeto, a exposição foi restrita ao necessário.

```
management:
  endpoints.web.exposure.include: health,metrics,prometheus
  endpoint.health.show-details: when-authorized
  endpoint.health.show-components: when-authorized
```

Endpoint	Uso
/actuator/health	Estado e componentes de saúde; detalhes apenas para autorizado
/actuator/metrics	Catálogo e inspeção pontual dos meters registrados
/actuator/prometheus	Formato coletado periodicamente pelo Prometheus

Segurança

Endpoints operacionais podem revelar topologia, dependências e comportamento interno. Exponha somente os necessários e aplique autenticação/autorização, rede privada ou ambos.

Actuator ≠ observabilidade completa

Ele expõe a superfície operacional e integra o Spring Boot com a infraestrutura. A coleta, o armazenamento, as consultas e os painéis pertencem aos backends.

5 · INSTRUMENTAÇÃO

Micrometer e a API Observation

Micrometer oferece uma camada neutra para instrumentar a aplicação. O código registra a intenção — contador, timer ou observação — sem ficar amarrado a um backend específico.

Peça	Função no projeto
MeterRegistry	Registra Counter, Gauge e Timer; o registry Prometheus traduz a saída
ObservationRegistry	Cria observações que podem produzir métricas e/ou spans
Tracer	Acessa span/contexto atual e permite criar spans explícitos
Propagator	Injeta e extrai traceparent/tracestate em transportes

```
Observation.createNotStarted("pedido.criar", observationRegistry)
    .observe(() -> criarPedidoComOutbox(dto));
```

```
Timer.builder("kafka.consumer.processamento.tempo")
    .tag("consumer", "estoque")
    .tag("resultado", "sucesso")
    .register(meterRegistry);
```

Separação saudável

Micrometer é a fachada usada pelo Spring; OpenTelemetry é o padrão/SDK que transporta traces; OTLP é o protocolo de exportação; Jaeger é o backend e a interface de análise.

6 · MÉTRICAS

Counter, Gauge, Timer e tags

O tipo do meter deve refletir o comportamento do dado. A escolha errada cria painéis enganosos.

Tipo	Semântica	Exemplo
Counter	Só cresce; calcule taxa ou aumento por janela	Falhas de publicação; mensagens enviadas à DLT
Gauge	Fotografia atual; pode subir e descer	Quantidade atual de eventos pendentes na Outbox
Timer	Duração + contagem; produz sum/count/buckets	Tempo de publicação e processamento Kafka

Tags e cardinalidade

- Boas tags têm conjunto pequeno e previsível: consumer=estoque|notificacao, resultado=sucesso|falha.
- IDs únicos — pedidoid, eventoid, traceld, usuário — não pertencem a labels de métricas.
- Cada combinação de labels vira uma série temporal. Cardinalidade alta aumenta memória, custo e lentidão.

Onde colocar identificadores únicos?

Use atributos de span para investigação causal e conteúdo do log para busca detalhada. Em Loki, mantenha esses IDs no texto, não como labels.

7 · IMPLEMENTAÇÃO

As métricas criadas no projeto

As métricas foram desenhadas para responder às perguntas operacionais da Outbox e dos consumers.

Meter Micrometer	Tipo	Pergunta respondida
outbox.eventos.pendentes	Gauge	Há fila acumulando agora?
outbox.publicacoes.falhas	Counter	Quantas tentativas falharam no período?
outbox.publicacao.tempo	Timer	Quanto tempo a publicação leva?
kafka.consumer.processamento.tempo	Timer + tags	Qual consumer/resultado está mais lento?
kafka.dlt.mensagens.enviadas	Counter	Quantas mensagens esgotaram as tentativas?

Nomes vistos pelo Prometheus

```
outbox_eventos_pendentes_eventos
outbox_publicacoes_falhas_tentativas_total
outbox_publicacao_tempo_seconds_sum
outbox_publicacao_tempo_seconds_count
kafka_dlt_mensagens_enviadas_mensagens_total
```

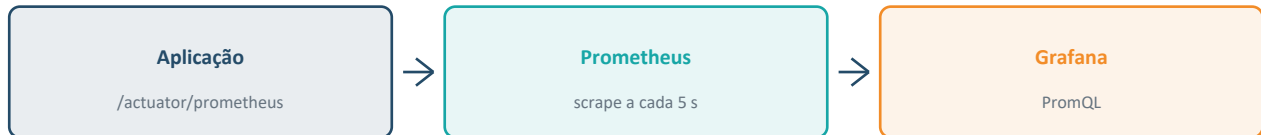
Por que há sufixos?

O registry Prometheus normaliza pontos para sublinhados e acrescenta unidade/sufixo conforme o meter. Timers expõem várias séries, como sum e count.

8 · COLETA

Prometheus e o modelo pull

O Prometheus visita o endpoint da aplicação em intervalos regulares. Ele armazena séries temporais compostas por nome, labels, timestamp e valor.



```
scrape_configs:
- job_name: projeto-springboot
  scrape_interval: 5s
  metrics_path: /ProjetoSpringBoot/actuator/prometheus
  static_configs:
  - targets: [host.docker.internal:8080]
```

PromQL usado nos painéis

Objetivo	Consulta
Pendentes atuais	outbox_eventos_pendentes_eventos
Falhas no período	sum(increase(outbox_publicacoes_falhas_tentativas_total[\$__range]))
Tempo médio	increase(..._sum[\$__range]) / increase(..._count[\$__range])
Aplicação disponível	up{job="projeto-springboot"}

9 · VISUALIZAÇÃO

Grafana: dashboards e exploração

O Grafana consulta as fontes configuradas. No projeto, Prometheus e Loki foram provisionados automaticamente para tornar o ambiente reproduzível.

Painel do dashboard	Decisão operacional
Eventos pendentes na Outbox	Ver acúmulo e confirmar se o job está drenando
Falhas de publicação no período	Detectar instabilidade no envio
Tempo médio até publicação	Acompanhar latência e degradação
Mensagens à DLT	Identificar eventos que esgotaram retries
Processamentos Kafka por resultado	Separar sucesso/falha por consumer
Tempo médio dos consumers	Comparar estoque e notificação

Dashboard vs Explore

Use o dashboard para perguntas recorrentes e visão rápida. Use o Explore para investigação livre, refinando PromQL ou LogQL durante um incidente.

Um bom painel

- tem título que expressa a pergunta, unidade correta e janela temporal explícita;
- mostra ausência de dados de forma consciente, sem confundir zero com falha de coleta;
- permite navegar do sintoma agregado para trace e logs correlacionados.

10 · LOGS

Correlação: traceId, spanId e eventId

O padrão de log coloca lado a lado a identidade técnica do trace e a identidade de negócio do evento.

```
correlation: >-  
[traceId=%X{traceId:-sem-trace}  
spanId=%X{spanId:-sem-span}  
eventId=%X{eventId:-sem-evento}]
```

Identificador	Escopo	Uso
traceId	Operação distribuída completa	Agrupar HTTP, Outbox, Kafka e consumers
spanId	Uma etapa do trace	Localizar o ponto exato que produziu o log
eventId	Evento de negócio	Reconhecer retries/redeliveries do mesmo evento

Por que aparece sem-evento?

A requisição já tem trace antes de existir um evento de negócio. O eventId passa a existir durante o registro da Outbox e é colocado no MDC ao publicar/consumir.

Por que aparece sem-trace?

Uma thread sem observação ativa não possui contexto no MDC. Após habilitar observação no Kafka e propagar traceparent, os consumers passaram a receber o trace corretamente.

11 · LOGS CENTRALIZADOS

Alloy, Loki e Grafana

A aplicação continua escrevendo em arquivo. O Alloy acompanha o arquivo, adiciona labels estáveis e envia os registros ao Loki.



Labels usados

```
service_name = "ProjetoSpringBoot"
environment = "local"
```

Consultas práticas

```
{service_name="ProjetoSpringBoot"}
{service_name="ProjetoSpringBoot"} |= " ERROR "
{service_name="ProjetoSpringBoot"} |= "traceId=12d87a..."
{service_name="ProjetoSpringBoot"} |= "eventoId=58c3d4f0..."
```

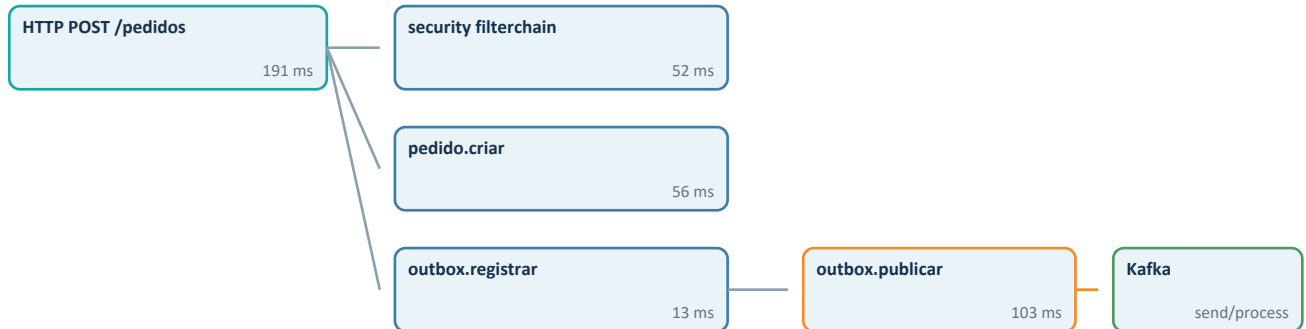
Cardinalidade no Loki

traceld e eventold ficam no conteúdo do log. Transformá-los em labels criaria um fluxo para quase cada mensagem, prejudicando índice e custo.

12 · TRACING

Trace, span, eventos e atributos

Um trace é o caminho causal completo. Cada unidade de trabalho é um span; spans formam uma árvore por relações de pai e filho.



Mesmo traceId; spanIds diferentes; lacuna assíncrona esperada

Elemento	Significado
traceId	Identidade do caminho completo; compartilhada por todos os spans
spanId	Identidade única de uma etapa
parentSpanId	Relação que monta a árvore causal
attribute/tag	Metadado pesquisável: evento.id, outbox.id, destination
event/log	Ocorrência temporal dentro do span, como uma exception
status	Resultado semântico, normalmente unset/ok/error

O spanId sempre muda?

Cada nova observação instrumentada cria outro spanId. O span HTTP é automático; pedido.criar, pedido.listar, outbox.registrar e outbox.publicar foram definidos por nós.

13 · ECOSSISTEMA

Micrometer × OpenTelemetry × OTLP × Jaeger

Esses nomes descrevem camadas diferentes da solução.

Camada	Papel	No projeto
Micrometer	API/fachada de observabilidade no ecossistema Spring	Meters, Observation, Tracer e Propagator
OpenTelemetry	Padrão e SDK vendor-neutral para telemetria	Implementação criada pelo starter do Spring Boot
OTLP	Protocolo para exportar traces/métricas/logs	HTTP para /v1/traces
Jaeger	Backend e UI de tracing	Recebe OTLP, armazena e exibe os traces

**Starter próprio**

spring-boot-starter-opentelemetry reúne dependências e autoconfiguração para criar o tracer/SDK e exportar via OTLP. Ele evita montar manualmente cada componente.

Configuração usada

```
management.tracing.sampling.probability: 1.0
management.tracing.export.otlp.enabled: true
management.otlp.tracing.endpoint: http://localhost:4318/v1/traces
```

14 · INSTRUMENTAÇÃO

Automática e explícita

O Spring cria spans em integrações suportadas; nós adicionamos spans onde a regra de negócio precisava de nomes e atributos próprios.

Origem	Spans observados
Automática HTTP	http post /pedidos
Spring Security	security filterchain before/after, authenticate bearer token, authorize request
Spring Kafka	pedidos-criados send e pedidos-criados process
Explícita no projeto	pedido.criar, pedido.listar, outbox.registrar, outbox.publicar

Span.kind encontrado no Jaeger

Kind	Interpretação
internal	Trabalho interno da aplicação, como outbox.publicar
producer	Envio ao broker
consumer	Processamento de uma mensagem recebida
server	Entrada HTTP atendida pelo servidor

Granularidade

Crie spans em fronteiras que ajudam a explicar latência, erro ou causalidade. Instrumentar cada método aumenta ruído e custo sem melhorar o diagnóstico.

15 · PROPAGAÇÃO

W3C Trace Context

O trace atravessa processos por headers padronizados. O principal é traceparent; tracestate transporta estado opcional de fornecedores.

```
traceparent: 00-12d87a799cf2e8fc3cf1f84a97c73aa6-c34f4c1934095a95-01
| | | |
versão traceId spanId flags
```

Campo	Tamanho	Significado
version	2 hex	Versão do formato
trace-id	32 hex	Identificador do trace completo
parent-id	16 hex	Span que representa o pai remoto
trace-flags	2 hex	Bit de sampling; 01 indica sampled

Contexto, não identidade de negócio

traceparent liga a telemetria. eventold continua sendo a chave de negócio usada para reconhecer o mesmo evento em retries, DLT e idempotência.

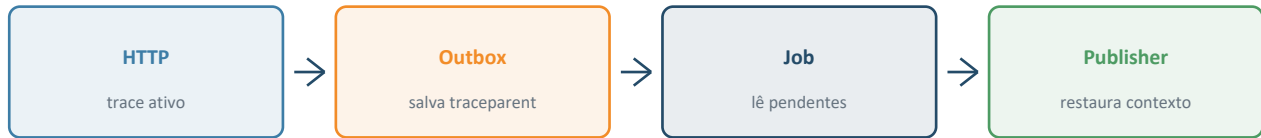
Sampling em estudo

A probabilidade 1.0 grava 100% dos traces para facilitar aprendizado. Em produção, o volume, o custo e a necessidade de retenção definem uma política menor ou adaptativa.

16 · OUTBOX

Como o trace atravessa a espera do job

A transação HTTP termina antes da publicação. Para preservar a causalidade, o contexto W3C foi salvo junto com o evento da Outbox.



Momento	O que acontece
Registrar evento	Propagator injeta traceparent/tracestate em carrier e a entidade persiste os valores
Job encontra lote	O job possui seu próprio trace técnico, pois o lote pode misturar origens
Publicar cada evento	Publisher extrai o contexto salvo e cria outbox.publicar como descendente do trace original
Enviar ao Kafka	Instrumentação injeta novo traceparent nos headers da mensagem

Por que o job tinha traceId diferente?

O span do agendamento representa a execução do lote, não um único pedido. Cada evento restaura seu trace de origem somente no momento da publicação. Isso evita atribuir todos os eventos ao trace arbitrário do job.

17 · KAFKA

Do producer aos dois consumers

Com observation-enabled no template e nos listeners, o Spring Kafka injeta e extrai o contexto automaticamente.

```
spring.kafka.template.observation-enabled: true  
spring.kafka.listener.observation-enabled: true
```



O que permanece igual

- Os consumers compartilham o mesmo traceId da requisição original.
- Cada envio/processamento tem spanId próprio porque representa uma etapa distinta.
- Os dois consumers são irmãos causais do envio e podem executar em paralelo.
- topic, partition, offset, consumer group e listener id aparecem como atributos automáticos.

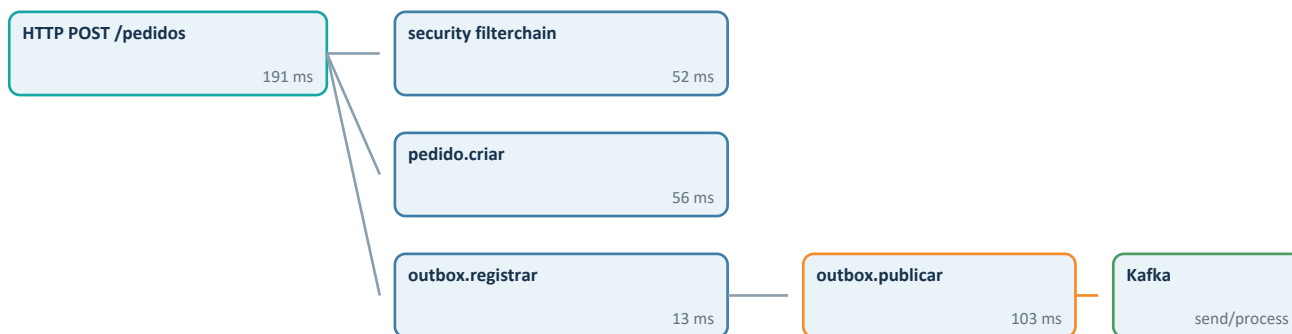
Identities complementares

topic + partition + offset identificam tecnicamente um registro no Kafka. eventId identifica o evento de negócio e continua útil se a mensagem mudar de tópico, for para DLT ou for reprocessada.

18 · JAEGER

Como ler um trace

Comece pela forma geral; depois aprofunde nos spans que concentram duração ou exibem erro.



Mesmo traceId; spanIds diferentes; lacuna assíncrona esperada

Visão	Como interpretar
Trace Timeline	Posição no tempo, duração, paralelismo e lacunas assíncronas
Trace Tabular	Lista compacta para ordenar/comparar operação, duração e spanId
Trace Graph	Grafo causal da execução observada; não é diagrama estático do código
Critical path	Segmentos que determinam a duração percebida do workflow
Tags	Atributos técnicos e de negócio do span
Logs	Eventos do span, incluindo exception.message e stacktrace
Process	Atributos do recurso/SDK que produziu a telemetria

Atenção ao termo Logs

No Jaeger, essa seção contém eventos do span. Ela não substitui os logs da aplicação armazenados no Loki.

19 · CASO NORMAL

Pedido criado com sucesso

O trace mostra o pedido respondendo rapidamente e a continuação assíncrona acontecendo alguns segundos depois.

Ordem	Etapas	Resultado
1	POST /pedidos + segurança	Trace criado automaticamente
2	pedido.criar	Regra de negócio executada
3	outbox.registrar	Pedido e evento persistidos na mesma transação
4	Resposta HTTP	Cliente recebe pedidold
5	Job encontra pendente	Trace próprio do lote
6	outbox.publicar	Contexto original restaurado
7	Kafka send + 2 process	Estoque e notificação concluem

191 ms × 4,73 s

A duração do span HTTP mede somente a resposta síncrona. A duração total do trace inclui a espera até o próximo ciclo do job e o processamento assíncrono. Nenhum número está errado; eles medem fronteiras diferentes.

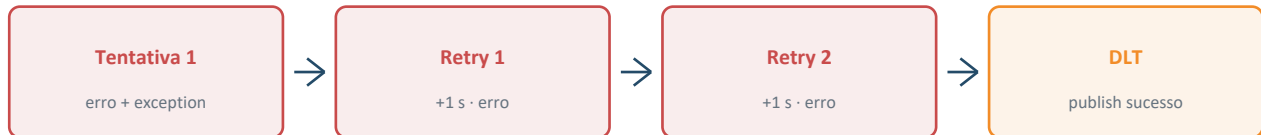
Leitura correta

- Barras curtas no início: trabalho síncrono.
- Espaço vazio: evento persistido aguardando o agendamento.
- Barras no fim: publicação e consumers retomando o mesmo trace.
- Branches paralelos: estoque e notificação são processamentos independentes.

20 · FALHA 999.99

Erro persistente, retries e DLT

O valor 999.99 aciona uma falha simulada no estoque antes da conclusão do efeito. O erro continua ocorrendo em todas as tentativas.



Sinal	Evidência esperada
Trace	Três spans consumer com error=true e eventos de exception
Jaeger Logs	exception.message = Falha simulada no processamento do estoque + stacktrace
Kafka	Mesmo topic/partition/offset em todas as tentativas
DLT	Span producer para pedidos-criados-dlt
Métrica	kafka.dlt.mensagens.enviadas incrementada
Outro branch	Consumer de notificação pode concluir normalmente

O envio à DLT não é outro erro

O span da DLT pode aparecer bem-sucedido: ele representa a recuperação após esgotar retries. A falha de negócio está nos spans anteriores do consumer.

Política configurada

FixedBackOff de 1 segundo com 2 retries significa 3 tentativas totais: a original + duas repetições.

21 · FALHA 888.88

Commit realizado, confirmação perdida

O valor 888.88 simula a janela mais perigosa: o efeito e o marcador de idempotência foram commitados, mas o consumer falha antes de confirmar o offset.



Tentativa	Banco	Kafka	Resultado
Primeira	Efeito e evento processado gravados	Offset não confirmado	Span com erro
Segunda	eventold já existe para estoque	Registro reenviado	Sem novo efeito; sucesso
Final	Estado permanece único	Offset confirmado	Sem DLT

O que procurar no trace

Mesmo traceId e eventold, mesmo topic/partition/offset, mas spanIds diferentes para cada tentativa. A segunda execução é uma nova etapa técnica sobre o mesmo evento de negócio.

Regra transacional

O efeito de negócio e o registro de idempotência precisam ser gravados na mesma transação. Separá-los recria uma janela em que um existe sem o outro.

22 · CONFIABILIDADE

As duas idempotências e a Outbox

São mecanismos complementares. Cada um fecha uma janela de duplicidade diferente.

Mecanismo	Protege contra	Não resolve sozinho
Idempotência do producer Kafka	Duplicação causada por retry interno/ack perdido no envio do producer	Redelivery e repetição do efeito no consumer
Idempotência do consumer	Executar novamente o mesmo evento de negócio	Perda do evento antes de chegar ao Kafka
Transactional Outbox	Pedido commitado sem evento e indisponibilidade temporária do Kafka	Duplicata após ack do Kafka e antes de atualizar status da Outbox

A janela clássica da Outbox



Sem promessa falsa de exactly-once

No conjunto, trabalhamos de forma robusta com entrega at-least-once: pode haver repetição técnica, mas o efeito de negócio permanece único por idempotência.

23 · DIAGNÓSTICO

Playbook: do alerta à causa

Use uma sequência repetível para evitar buscas aleatórias durante um incidente.

Passo	Ação	Ferramenta
1	Confirme disponibilidade e janela do problema	Grafana / up
2	Veja pendentes, falhas, DLT e tempos	Dashboard Prometheus
3	Escolha um trace lento ou com erro	Jaeger Search
4	Localize critical path, error span e exception	Jaeger Timeline
5	Copie traceId ou eventId	Tags/log correlation
6	Busque a narrativa completa	Grafana Explore / Loki
7	Valide retry, DLT, deduplicação e recuperação	Jaeger + métricas + logs

Atalhos úteis

```
# Logs de um trace
{service_name="ProjetoSpringBoot"} |= "traceId=<id>"

# Logs de todas as tentativas do evento
{service_name="ProjetoSpringBoot"} |= "eventId=<uuid>"

# DLT no intervalo visível do Grafana
increase(kafka_dlt_mensagens_enviadas_mensagens_total[$__range])
```

Finalize com evidência

Depois da correção, repita o cenário e confirme nos três sinais: contador estabilizado, trace concluído sem erro inesperado e logs coerentes.

24 · ARMADILHAS

Erros comuns e limites de produção

A implementação de estudo usa 100% de sampling e ambiente local. Produção exige políticas explícitas de volume, segurança e retenção.

Armadilha	Consequência	Prática recomendada
IDs únicos como labels	Explosão de cardinalidade	IDs em span/log; tags limitadas em métricas
Trace sem contexto na Outbox	Fluxo quebrado no job	Persistir e restaurar W3C trace context
Marcar job inteiro com um trace de pedido	Causalidade incorreta em lotes	Trace do job separado; contexto por evento
Efeito e dedupe em transações distintas	Duplicidade ou evento marcado sem efeito	Mesma transação local
Sampling 1.0 permanente	Custo e volume excessivos	Política compatível com tráfego/risco
Actuator aberto	Exposição operacional	Menor superfície + autenticação/rede
Logs sem retenção	Disco/custo descontrolado	Rotação, limites e política de retenção

PII e segredos

Não registre tokens, credenciais, dados pessoais ou payloads sensíveis em tags, logs ou atributos de span. Telemetria também precisa de governança.

25 · CHECKLIST

Ficha rápida de verificação

Use esta página ao revisar uma nova integração ou antes de demonstrar o ambiente.

Fundamentos

- [] Sei diferenciar métricas, traces e logs e quando usar cada sinal.
- [] Sei explicar Actuator, Micrometer, OpenTelemetry, OTLP, Jaeger e Grafana.

Métricas

- [] Counter, Gauge e Timer representam a semântica correta.
- [] Labels são limitadas e não contêm IDs únicos.
- [] Prometheus coleta /actuator/prometheus e o dashboard mostra dados.

Tracing

- [] HTTP, Outbox, Kafka producer e consumers compartilham o trace original.
- [] traceparent é persistido/restaurado na Outbox e propagado no Kafka.
- [] Spans têm nomes úteis, atributos seguros e erros registrados.

Confiabilidade

- [] Retries têm limite e backoff; após esgotar, a mensagem vai à DLT.
- [] Consumer deduplica por eventid + consumer dentro da transação do efeito.
- [] Entendo a janela de duplicidade entre ack do Kafka e status da Outbox.

Logs

- [] Arquivo é coletado pelo Alloy e consultável no Loki.
- [] traceid/spanid/eventid aparecem no texto; labels permanecem estáveis.

Critério de conclusão

Você consegue partir de uma anomalia no dashboard, encontrar o trace, localizar a exceção e reunir todos os logs do mesmo evento.

26 · GLOSSÁRIO

Vocabulário essencial

Termo	Definição curta
Meter	Instrumento de métrica registrado no Micrometer
Observation	Unidade observável que pode acionar handlers de métricas e tracing
Series	Nome de métrica + conjunto de labels ao longo do tempo
Scrape	Coleta pull feita pelo Prometheus
Trace	Caminho causal completo de uma operação
Span	Etapa temporizada dentro do trace
Context propagation	Transporte do contexto entre processos/threads
OTLP	Protocolo de exportação do OpenTelemetry
Sampling	Decisão de registrar/exportar uma fração dos traces
Outbox	Evento persistido na mesma transação da mudança de negócio
Retry	Nova tentativa do mesmo processamento
DLT	Tópico de destino após esgotar a política de retries
Idempotência	Repetir uma operação sem repetir seu efeito de negócio
Cardinalidade	Quantidade de combinações distintas de labels
LogQL	Linguagem de consulta do Loki
PromQL	Linguagem de consulta do Prometheus

27 · REFERÊNCIAS

Documentação oficial e implementação consultada

As referências abaixo consolidam os conceitos e permitem aprofundar cada componente.

Documentação oficial

- Spring Boot · Observability — <https://docs.spring.io/spring-boot/reference/actuator/observability.html>
- Spring Boot · Tracing — <https://docs.spring.io/spring-boot/reference/actuator/tracing.html>
- Spring Boot · Actuator Endpoints — <https://docs.spring.io/spring-boot/reference/actuator/endpoints.html>
- Spring Boot · Metrics — <https://docs.spring.io/spring-boot/reference/actuator/metrics.html>
- Micrometer Reference — <https://docs.micrometer.io/micrometer/reference/>
- OpenTelemetry · Traces — <https://opentelemetry.io/docs/concepts/signals/traces/>
- OpenTelemetry · Context propagation — <https://opentelemetry.io/docs/concepts/context-propagation/>
- W3C Trace Context — <https://www.w3.org/TR/trace-context/>
- Prometheus · Overview — <https://prometheus.io/docs/introduction/overview/>
- Grafana · Data sources — <https://grafana.com/docs/grafana/latest/datasources/>
- Grafana Loki · Getting started — <https://grafana.com/docs/loki/latest/get-started/>
- Grafana Alloy · Logs from files — <https://grafana.com/docs/alloy/latest/monitor/monitor-logs-from-file/>
- Jaeger · Deployment — <https://www.jaegertracing.io/docs/1.76/deployment/>

Arquivos centrais do ProjetoSpringBoot

pom.xml · application.yaml · application-docker.yaml · compose.yaml · prometheus.yml · Alloy/Loki configs · OutboxMetrics.java · KafkaProcessamentoMetrics.java · DltMetrics.java · EventoOutboxService.java · PublicadorEventoOutboxService.java · EstoquePedidoConsumer.java · KafkaConsumerConfig.java · DltObservabilityRecoverer.java

Escopo desta ficha

O documento descreve o ambiente efetivamente estudado em 31/08/2026. Configurações, versões e políticas devem ser revisitadas antes de produção.